

- Going back to bbst T ; We want, at all times, $ht(T) \leq O(\lg |T|)$.

- There are two famous ds that achieve this:

- (i) AVL tree [Adelson-Velsky, Landis, 1962]
- (ii) RB (Red-Black) tree [Guibas, Sedgwick, 1978]

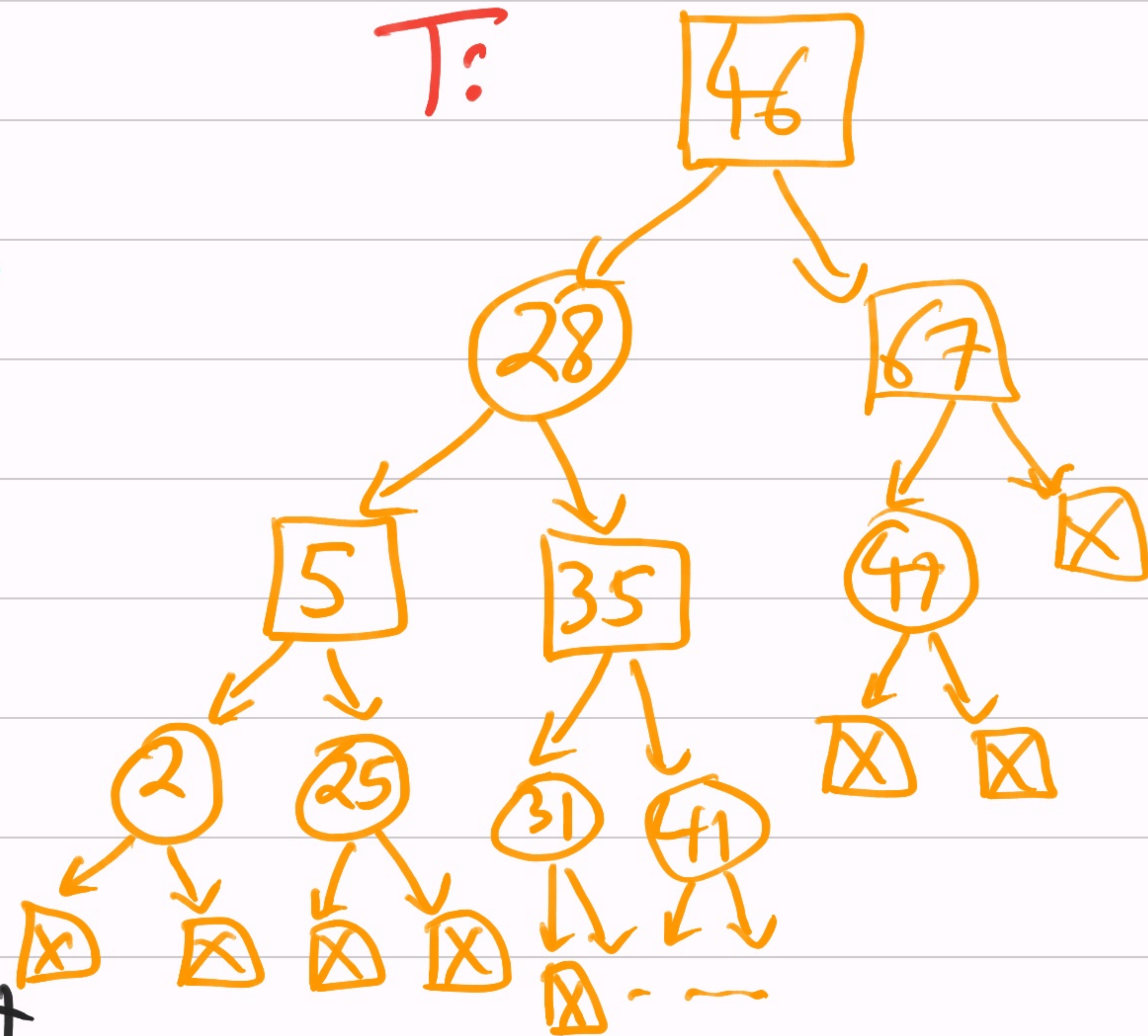
- We'll cover RB trees, because they're much faster, on average, on insert/delete.

Defn: RB tree T is a BST s.t.

- (i) each node is Red- $\bigcirc$ or Black- $\square$
- (ii) each leaf is Black- $\square$ denoted $\boxtimes$.
- (iii) each Red- $\bigcirc$ node has only Black- $\square$ children.
- (iv) # Black- $\square$ nodes on each path, from the root to any leaf, is the same. It is called black-height $bh(T)$.

e.g.

T:



Lemma: Any RB tree T
has height $\leq$
 $O(\lg |T|)$.

$$\triangleright \frac{h(T)}{2} \leq b h(T) \leq h(T) + 1.$$

[Pf. Two red nodes cannot
appear successively in a path. $\square$]

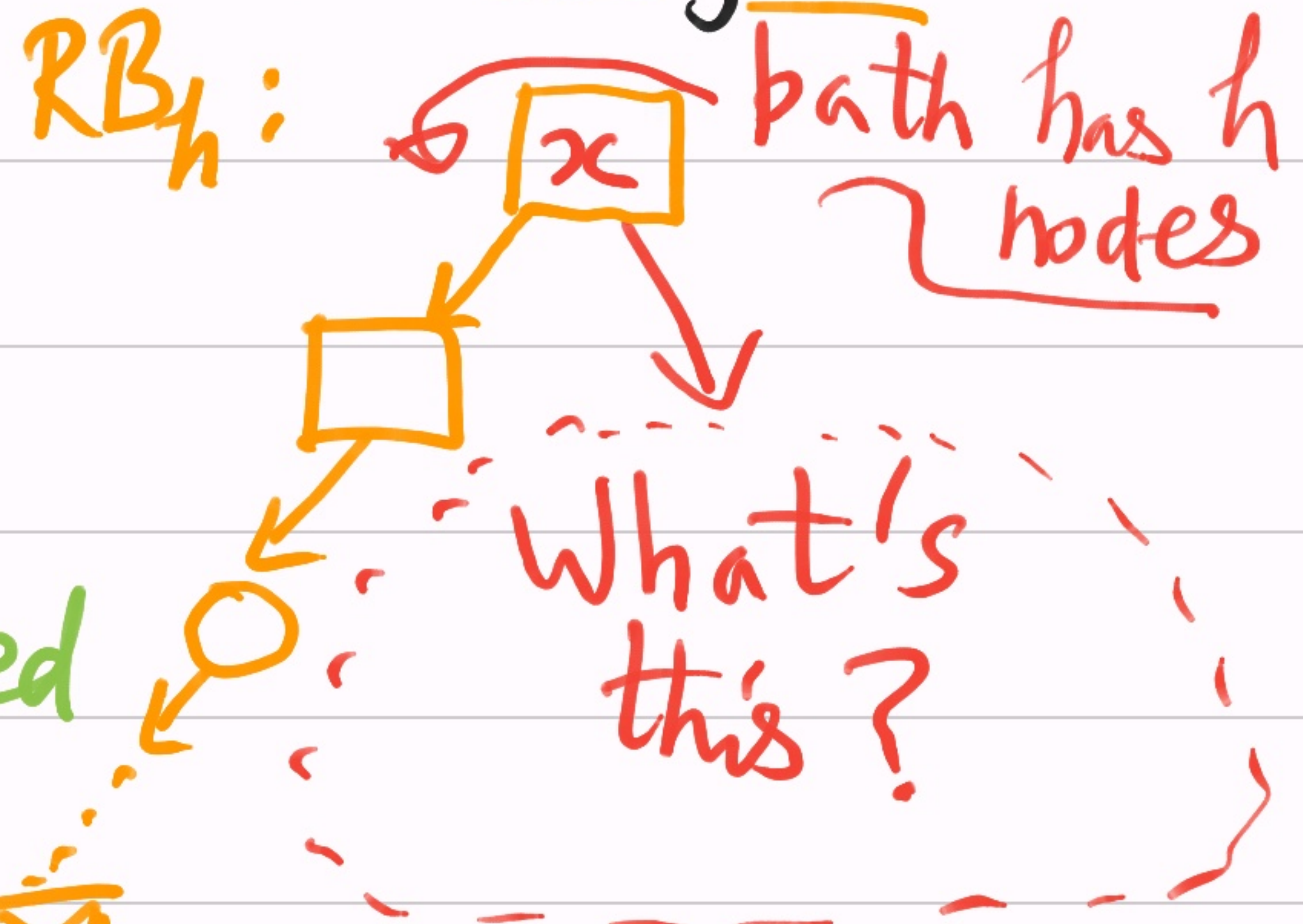
Pf. of Lemma : • Let m_h be the smallest RB tree RB_h with the longest-path having h nodes.

- W.l.o.g. RB_h has its left path the longest possible, i.e. having h nodes.

- This path has max. possible R-O nodes. [Else, the bh

R-O nodes. [Else, the bh increases and that has to be maintained for all other leaves. $\Rightarrow$ layer size.]

$\Rightarrow$ It has $\lfloor \frac{h+1}{2} \rfloor$ R-O & $\lfloor \frac{h-1}{2} \rfloor$ B- $\square$ nodes.
 (red on a path) = $b_h(RB_h)$



$\Rightarrow$ The right subtree(s) of this path should have only Black- $\square$ nodes as: we want to

(i) minimize the #nodes, and

(ii) all paths to leaves have $bh = \lfloor \frac{h-1}{2} \rfloor$.

$\Rightarrow$ rt subtree (x) is a complete binary tree of $ht = bh(RB_h) = \lfloor \frac{h-1}{2} \rfloor$. ($\triangleright RB_h$ has R-O root iff h is odd.)

$\Rightarrow$

$$m_h = |left\ size| + |root| + |right\ size| \\ = m_{h-1} + 1 + (2^{\lfloor \frac{h-1}{2} \rfloor} - 1).$$

$$\Rightarrow m_h = 2^{\lfloor \frac{h+1}{2} \rfloor} + 2^{\lfloor h/2 \rfloor} - 2 \Rightarrow h \leq O(\lg m_h). \square$$

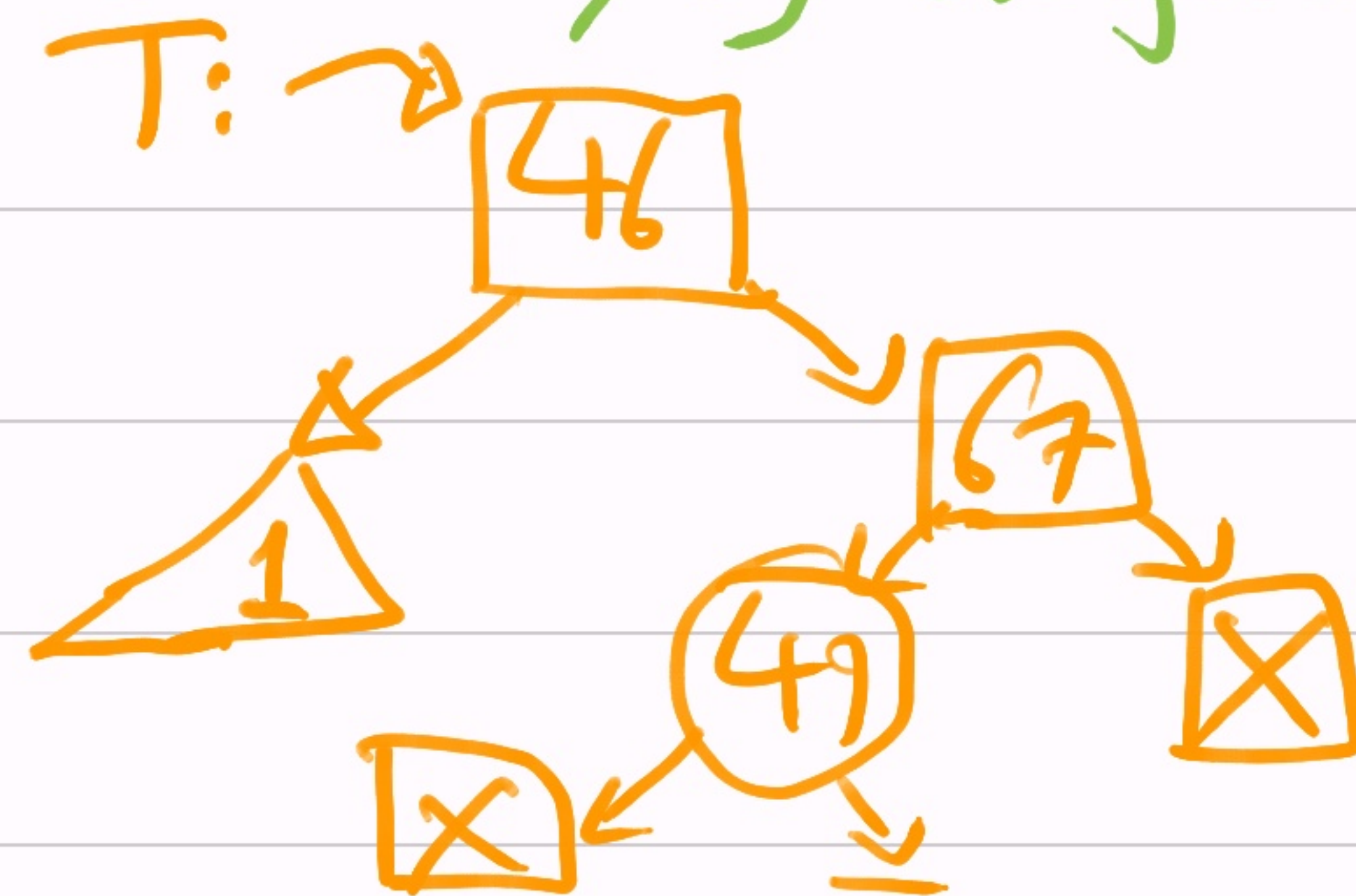
Exercise: Yet, RB tree could be unbalanced.

– Insertion ideas: BST-insert would violate one of the RB tree axioms.

Correct by: (i) recolor, or
(ii) rotate a subtree.

→ Assign a new node Red- $\bigcirc$ color, by default.

(i) – e.g. $\text{insert}(83, T)$:
• Create $\textcircled{83}$, search and insert.



→ get a new RB tree.

(ii) e.g. $\text{insert}(54, T)$.

⇒ color imbalance.

Resolution:

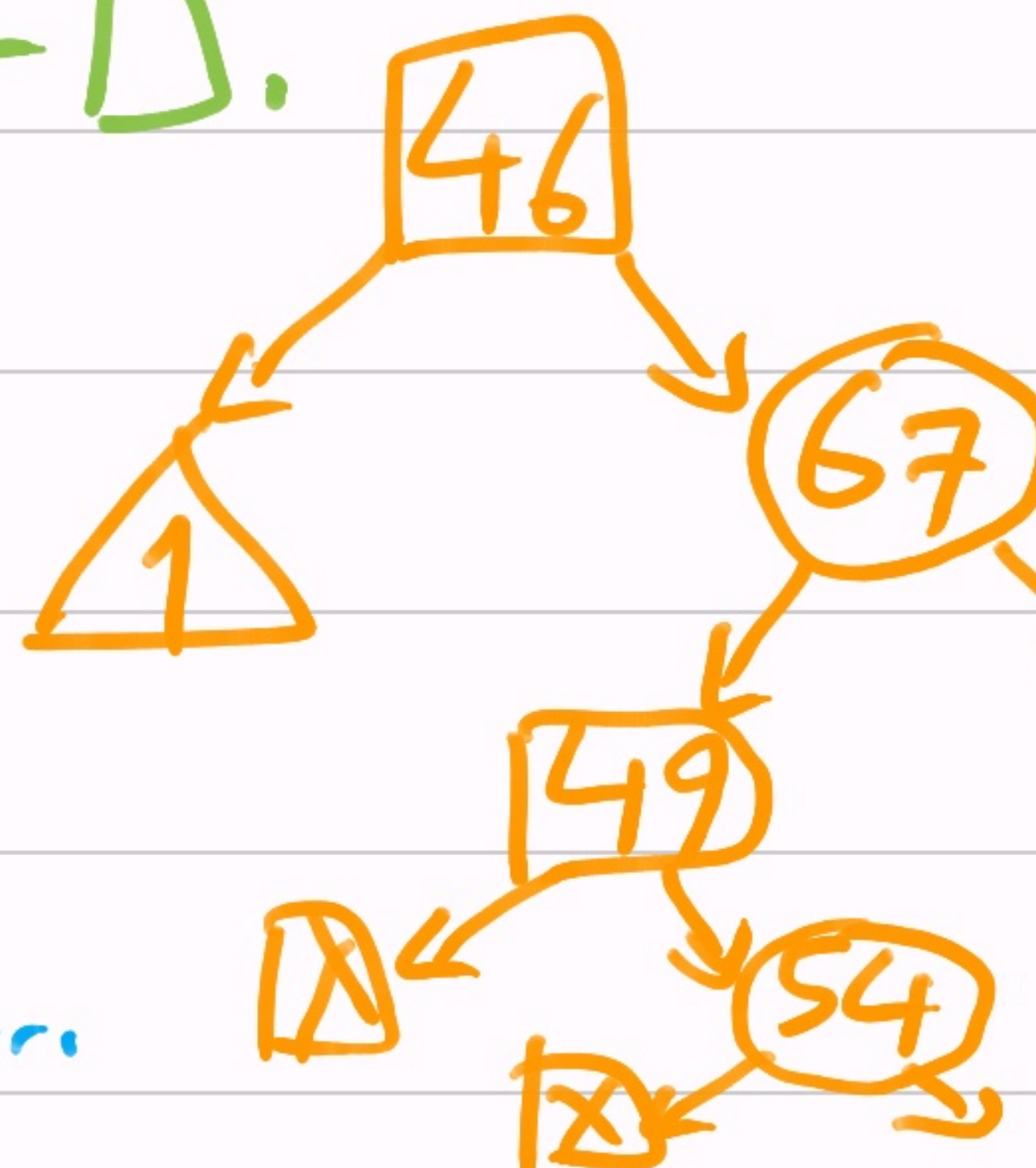
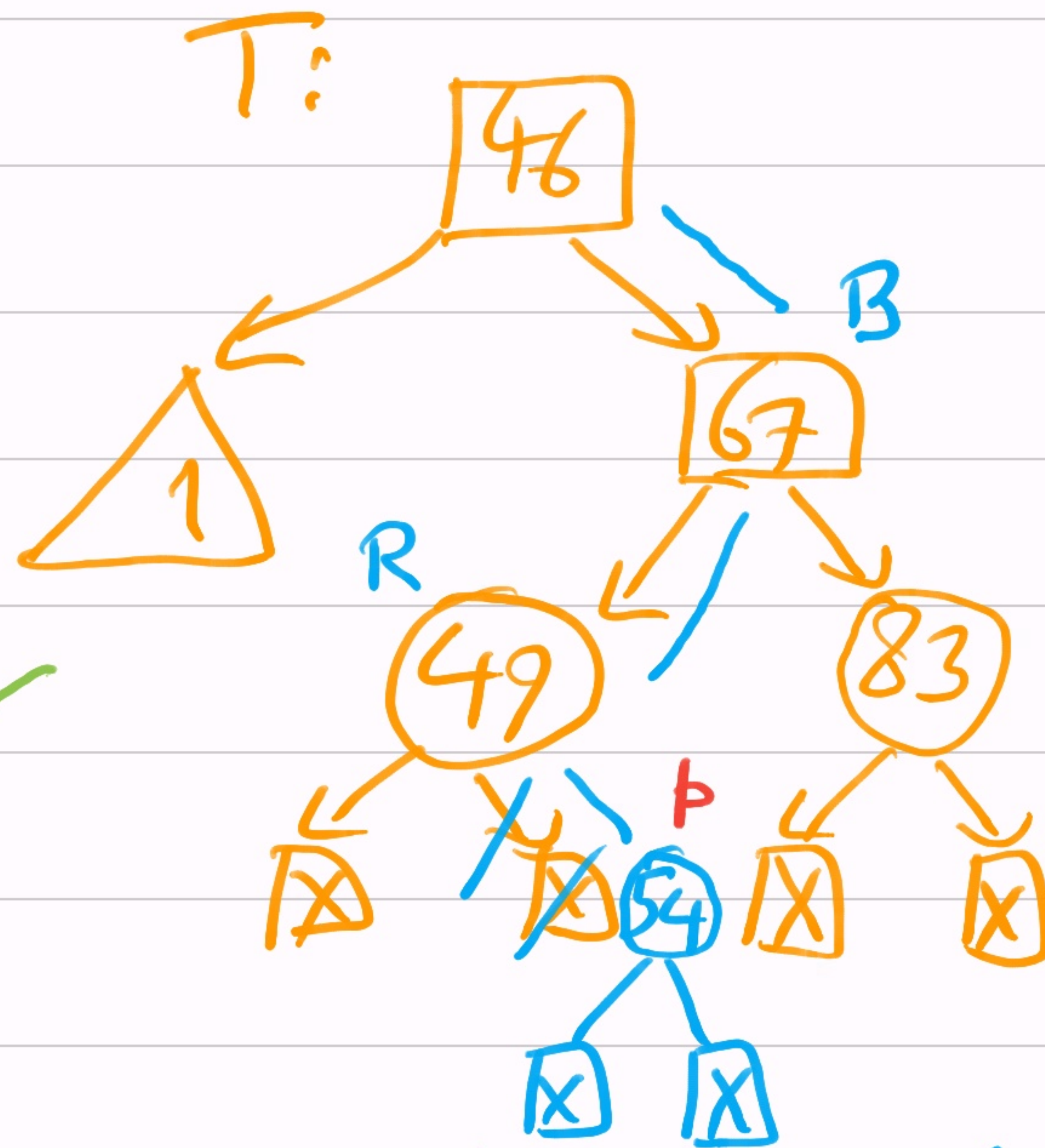
- make 49 B- $\square$.
- make 67 R-O.
- make 83 B- $\square$.

• This solves the cases:

- $\text{par}(p) \& \text{uncle}(p) = \text{R-O}$.

- $\text{par}(p) = \text{B-}\square$.

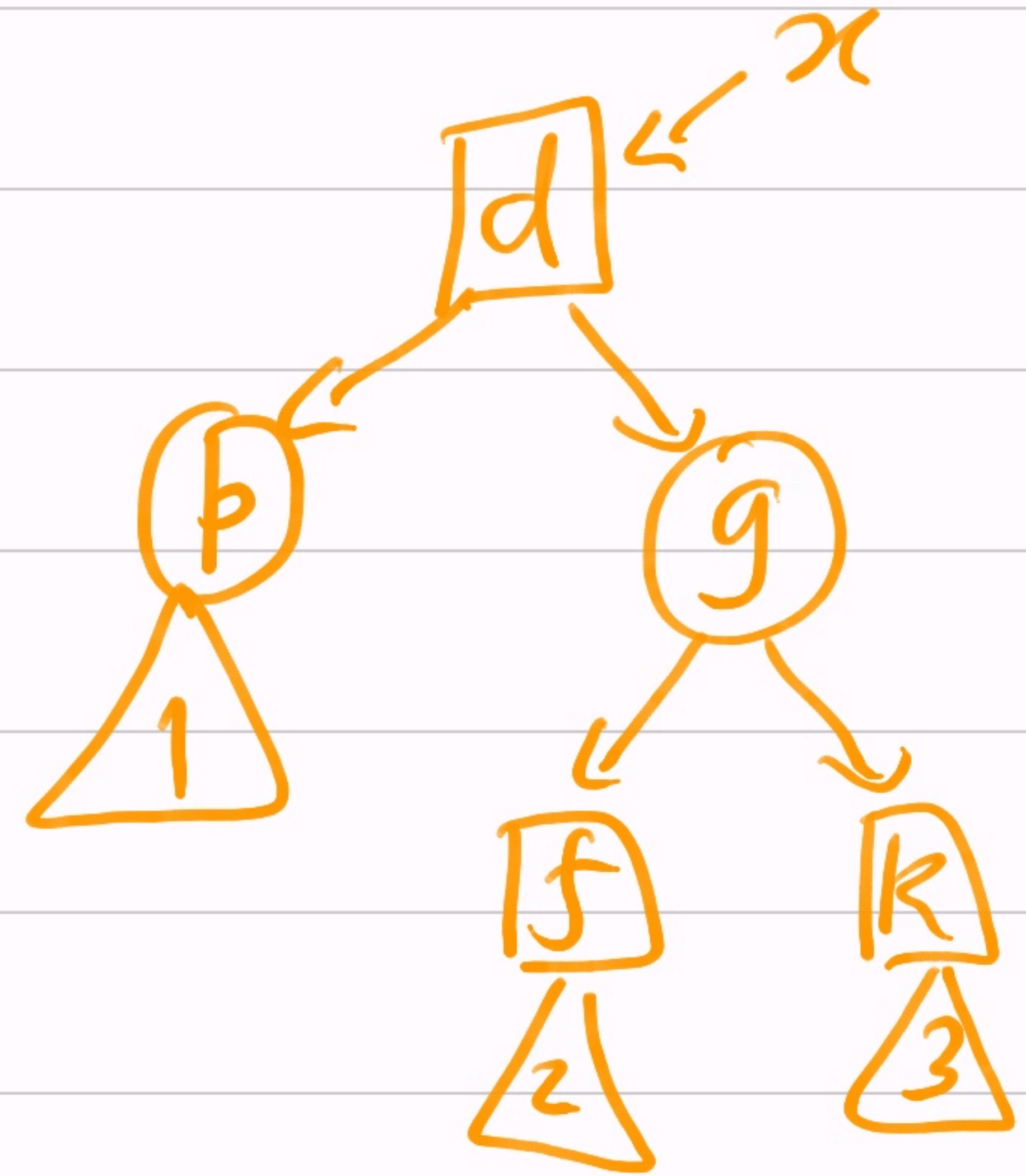
[Move upwards if $\text{gr.par}(p) = \text{R-O}$



...creates an imbalance.
So, we might reach the root & make it B- $\square$.]

▷ T_{new} is RB tree and
it took $O(\text{ht}(T))$ time.
 $\leq O(\lg |T|)$.

T_{new} :



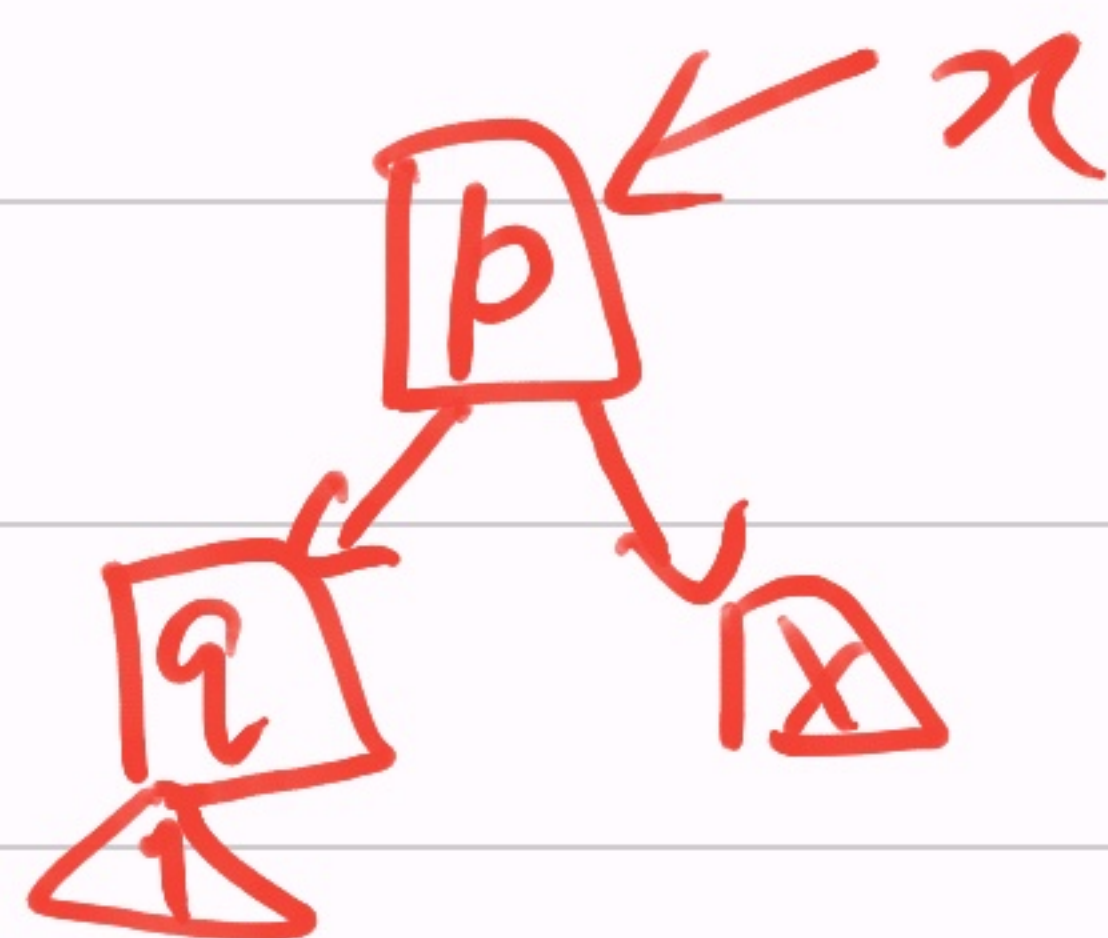
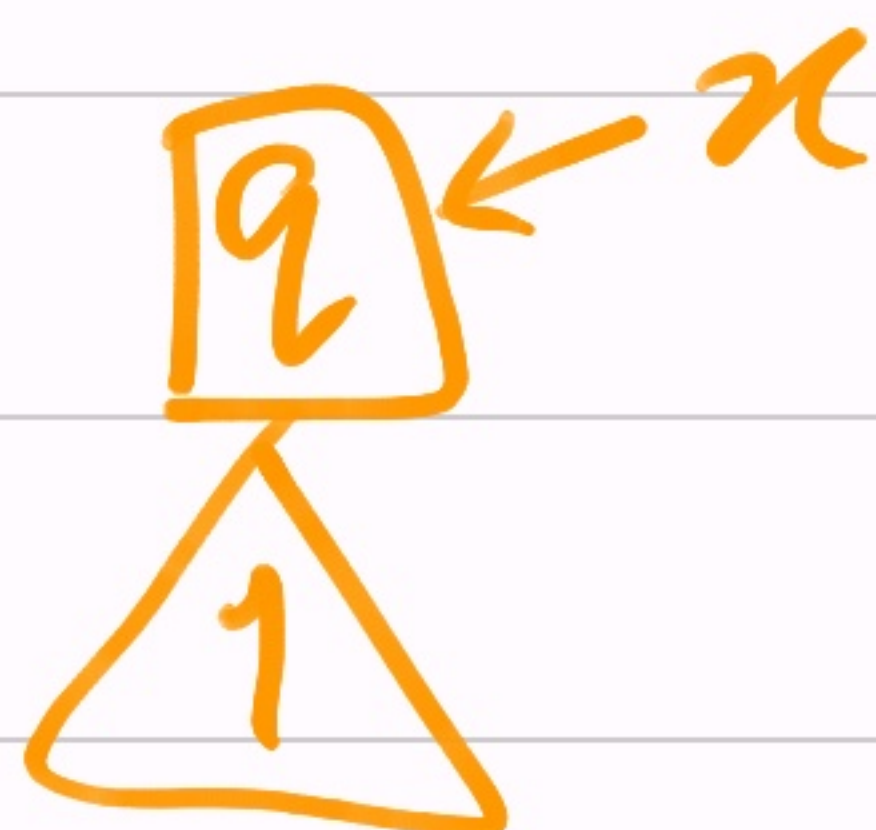
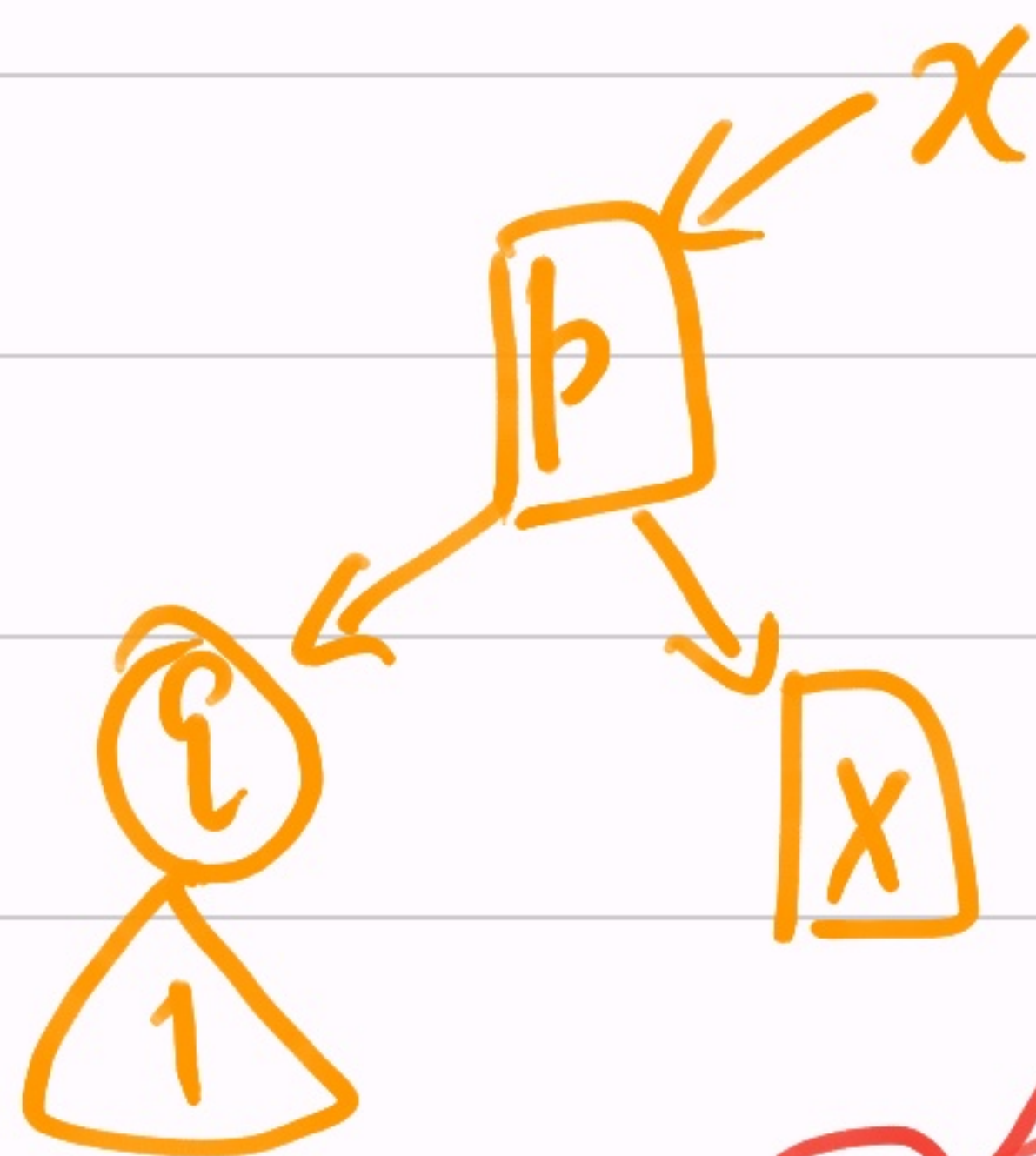
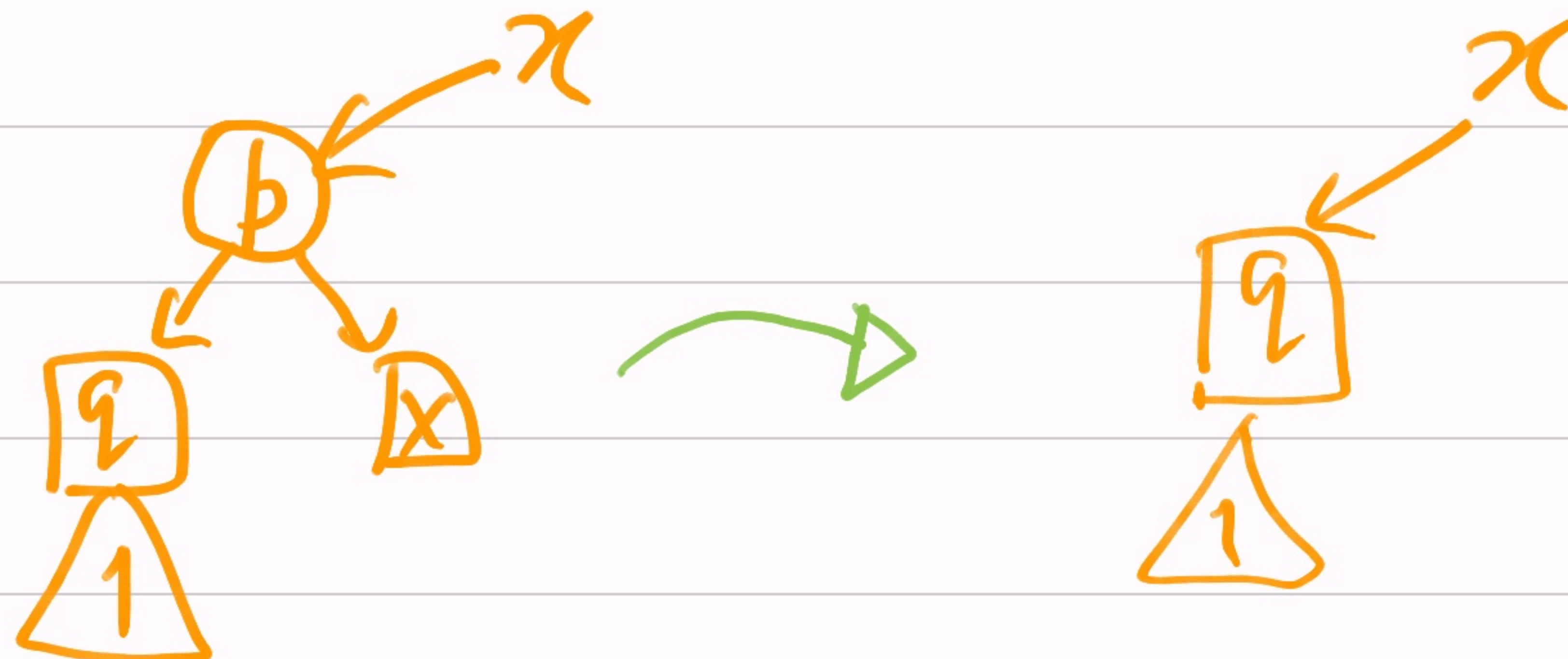
Exercise: Handle the (symmetric)
pseudocode when p appears
on the right side.

[Hint: Use a left-rotation and reduce
to the previous case.]

is RB-tree!

Deletion in RB tree T :

- e.g. $\text{delete}(p, T)$



denoted by $(-)$
uniform deficiency
in lh of x -subtree.

— We'll resolve this bh-deficiency later. First, we show: $\triangleright$ $\text{delete}(p, T)$ reduces to the case where p has at most 1 non-NULL child.

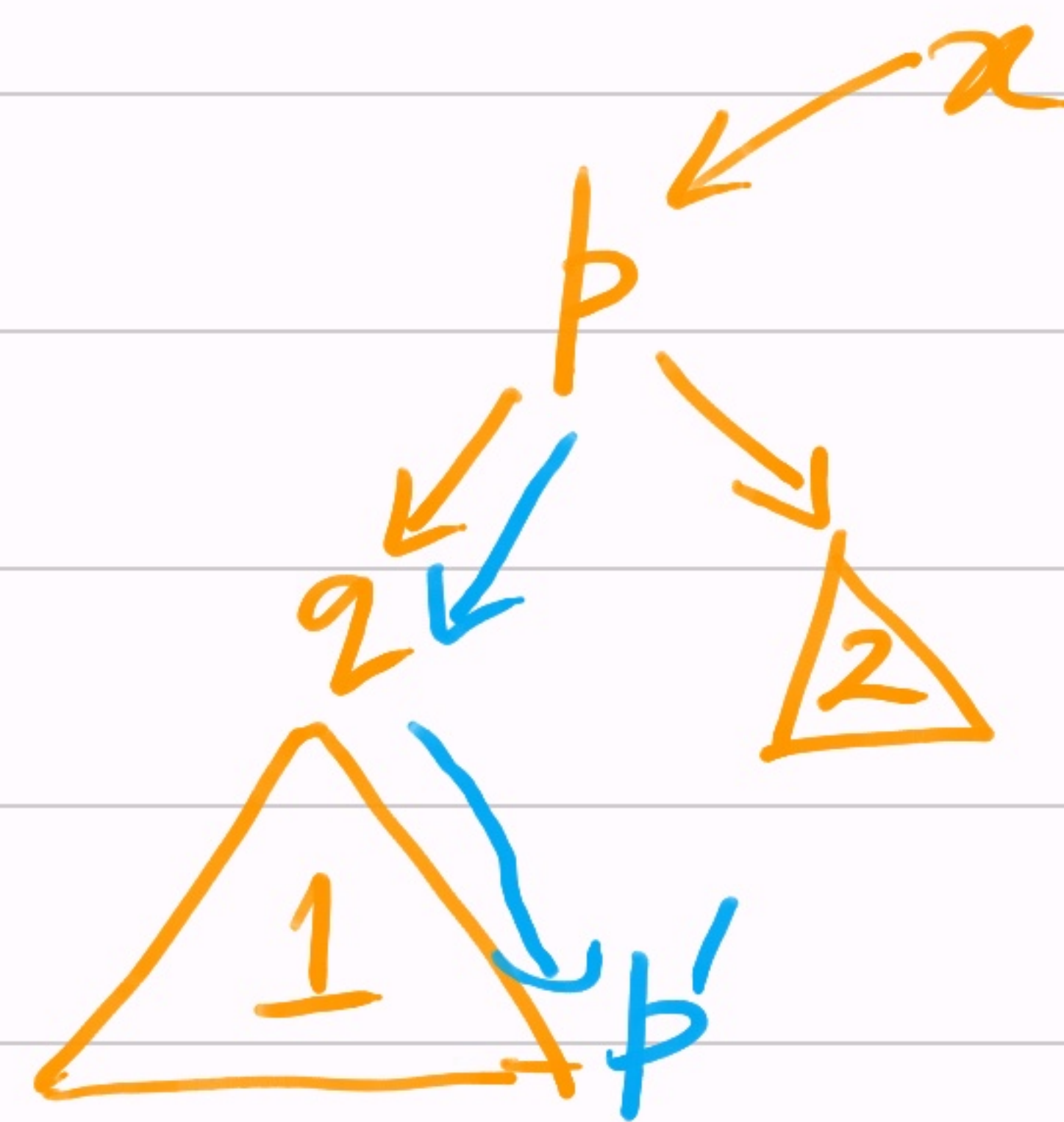
Pf: • Find the predecessor p' of p .

• Swap their keys.

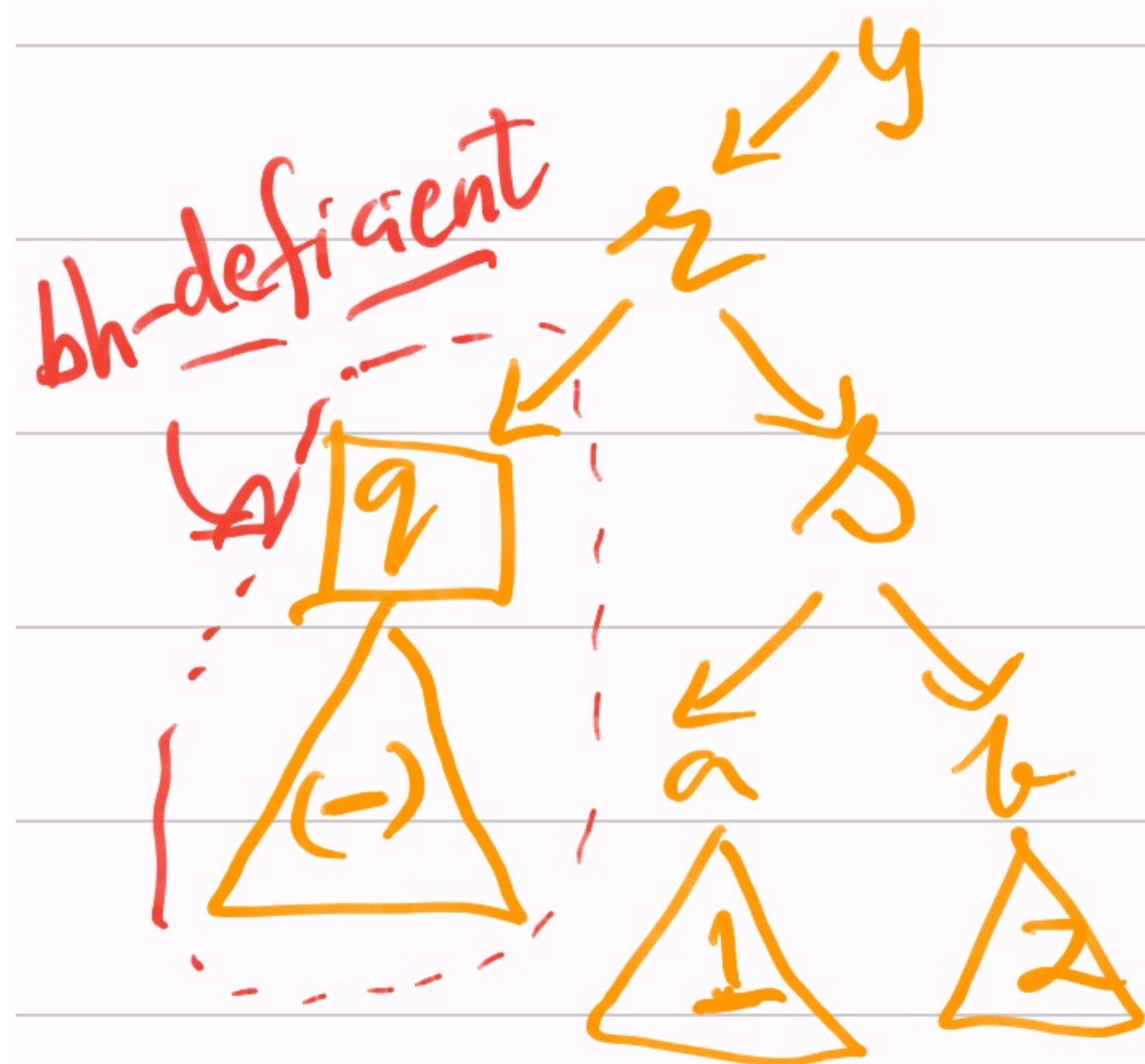
• p' has ≤ 1 non-NULL child!
($\because p'$ can't have "rt.child.")

• $\text{delete}(p', T)$.

□

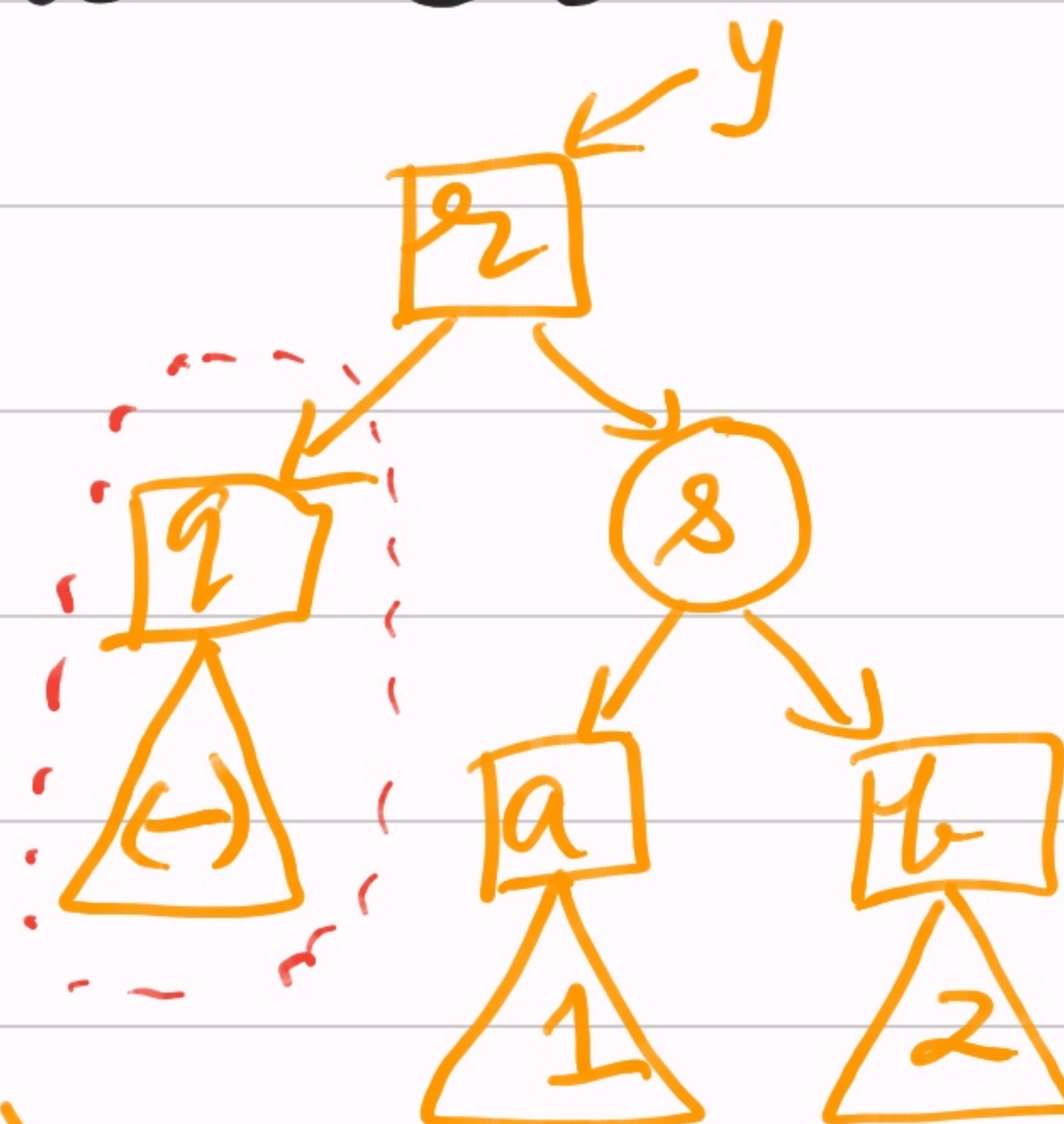


— Now we can move to the bh-deficient case:

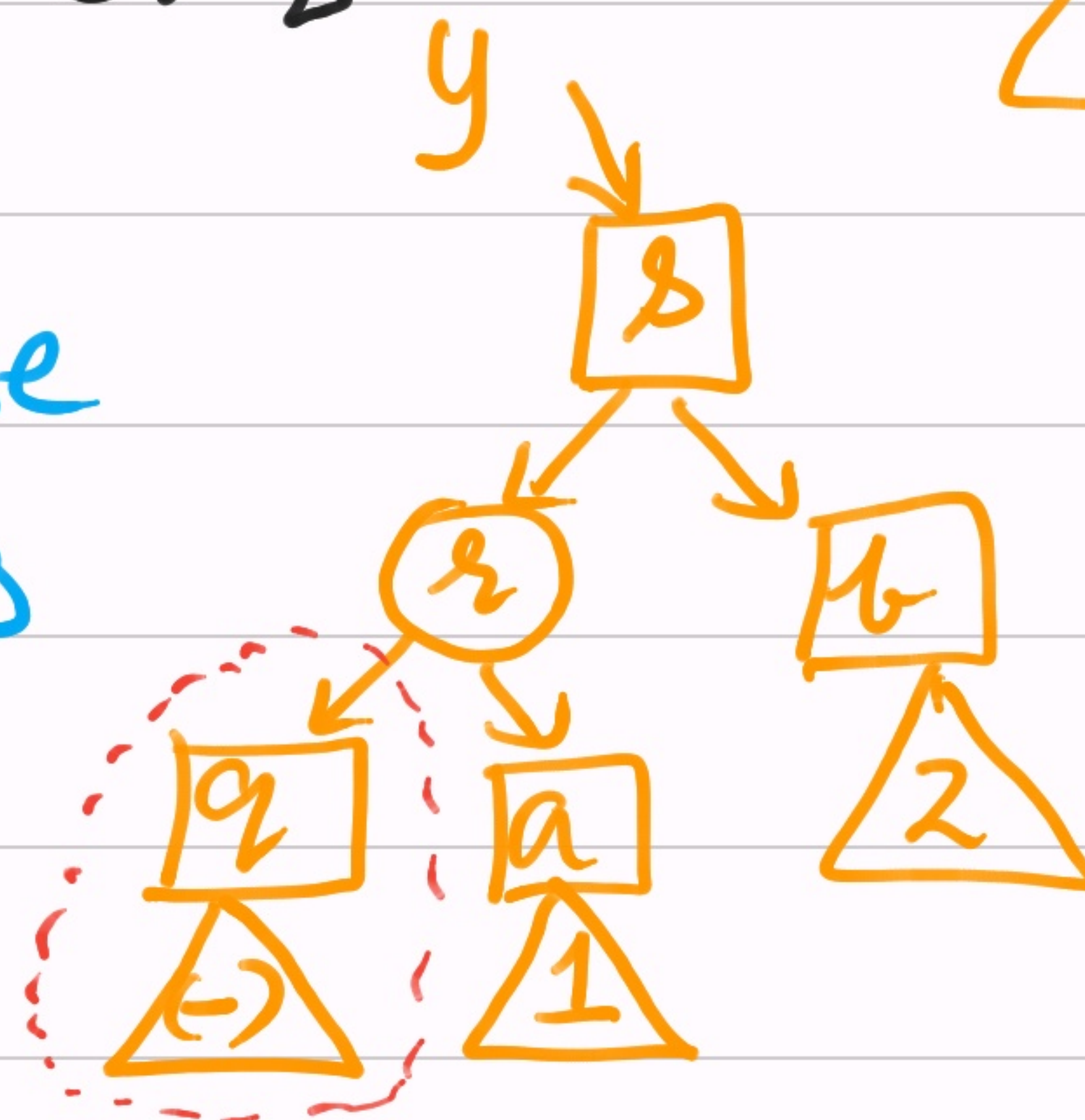


Case [q 's sibling s is R-O]:
 $\Rightarrow r$ is B- $\square$.

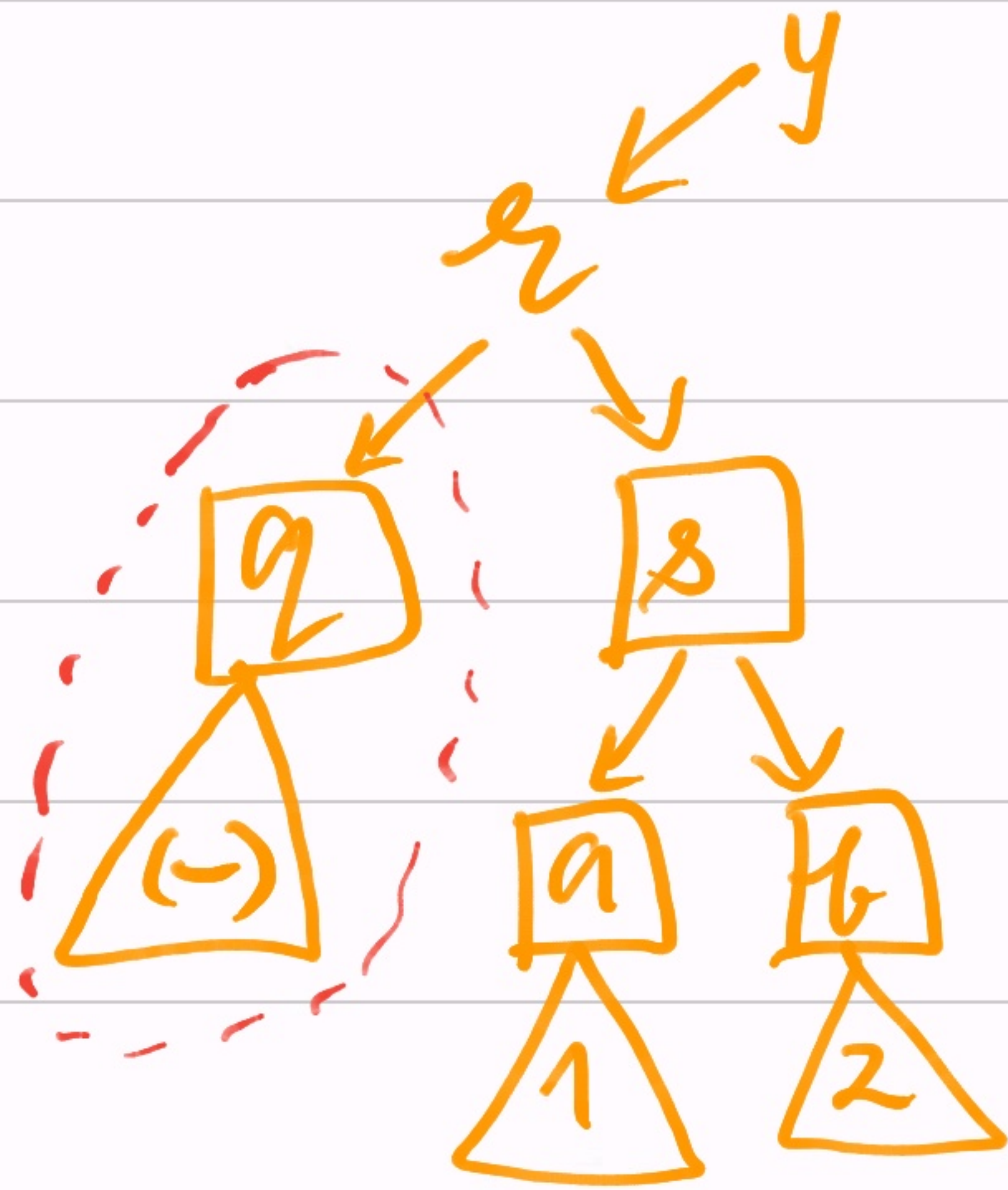
• To resolve the left deficiency, let's left-rotate around r & swap colors: $\rightarrow$



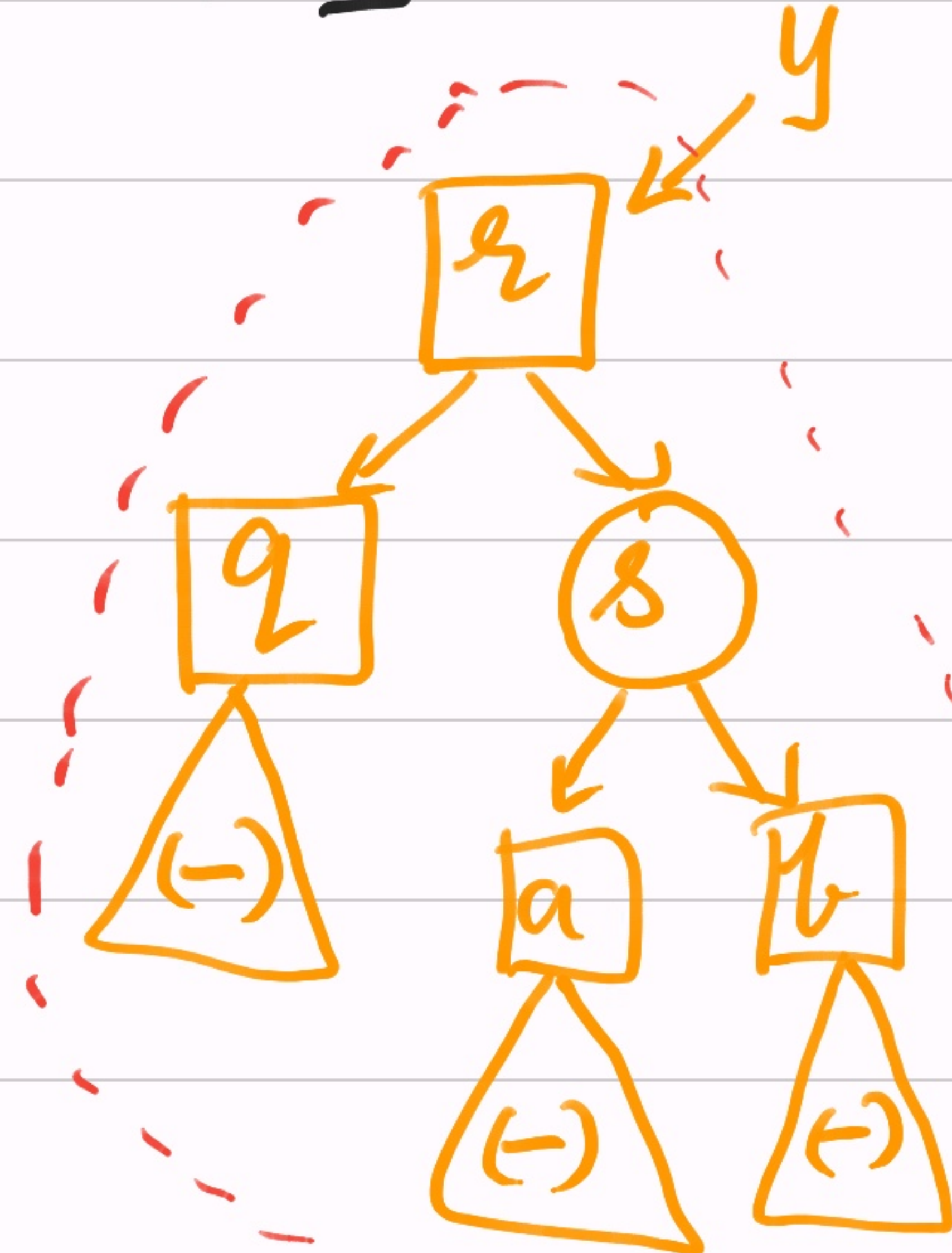
$\Rightarrow$ We've reduced to the case where bh-deficient node q has sibling (here 'a') Black- $\square$.



Case [q 's sibling s is B- $\square$, with both children also B- $\square$.]



- Make $\underline{r}$ B- $\square$.
 - Make $\underline{s}$ R- $\square$.
-

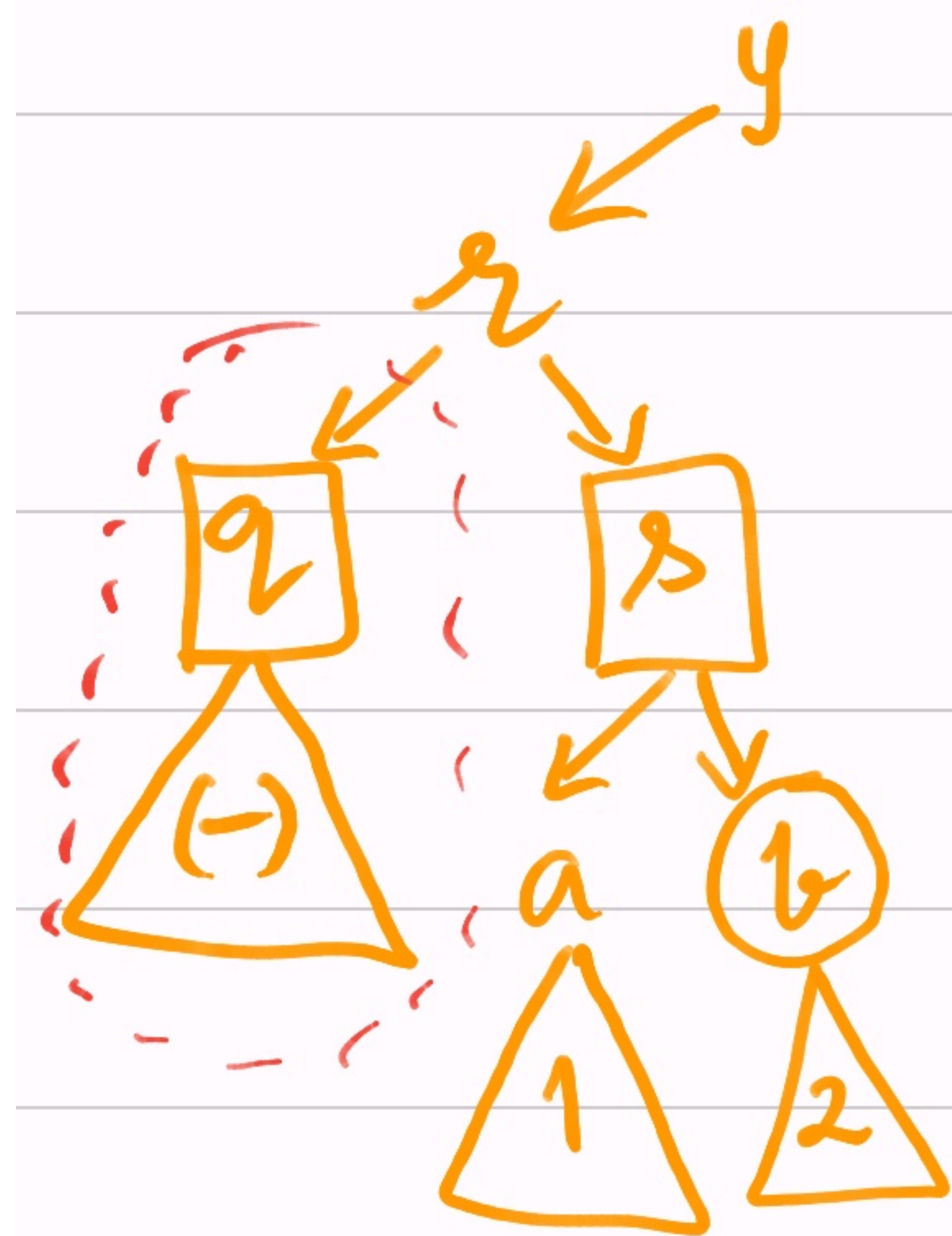


The bh-
 $\Rightarrow$ deficiency
 propagates
 above to y .

$\Rightarrow$ Ultimately, we reach the root: at that point the bh-deficiency is no more an issue!

Case [q 's sibling s is B- $\square$, with some child R- $\circ$]

- Due to a possible right-rotation, we can assume w.l.o.g. that the right-child of s is R- $\circ$.

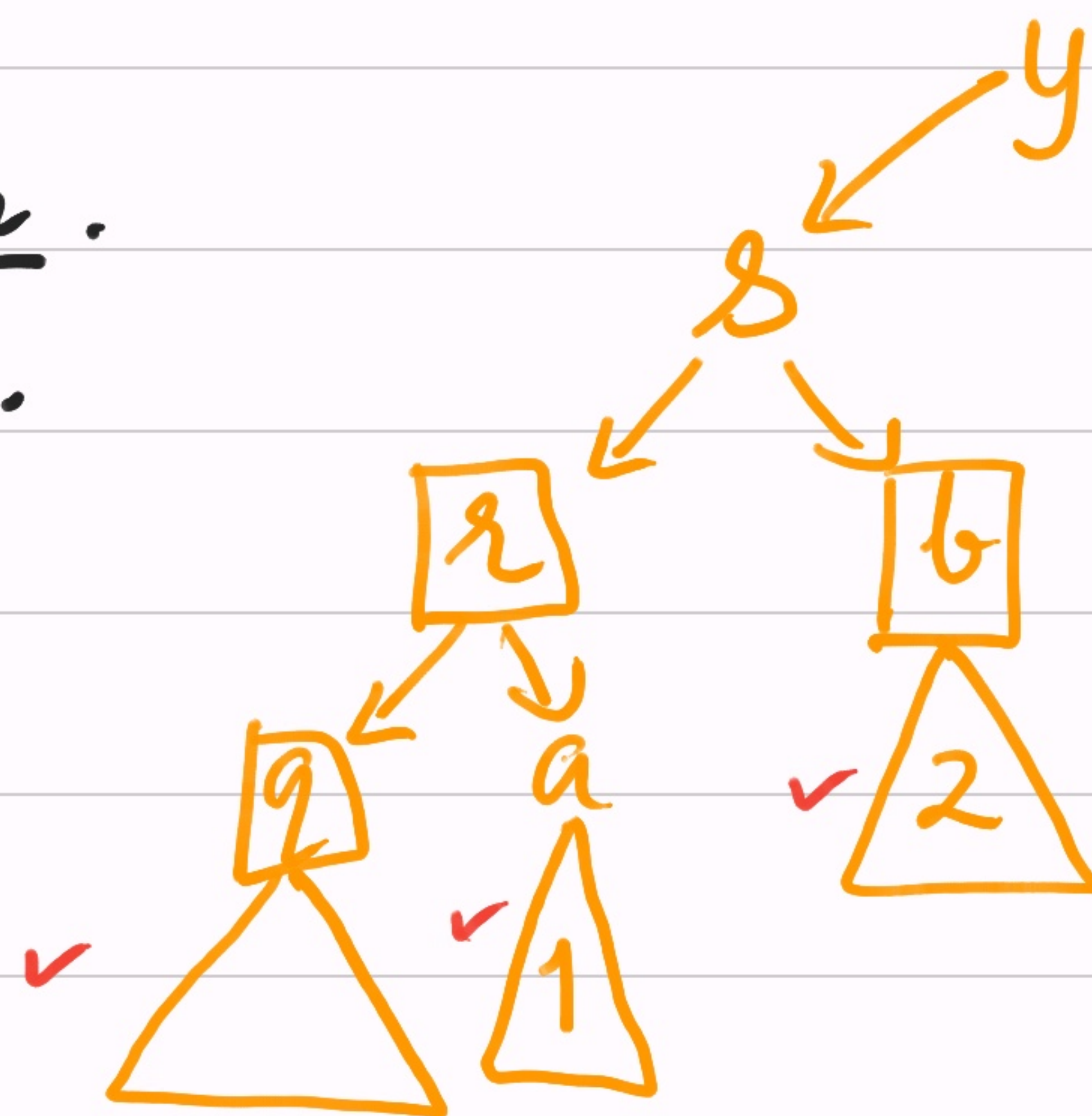


- Left-rotate around r .

- swap r , s colors.

→

- make b B- $\square$.



⇒ no more deficient anywhere!

⇒ We've handled all the cases of deletion.

▷ Delete takes $O(\text{ht}(T)) \leq O(\lg |T|)$ time in an RB tree T .

- We can now collect all the operations that we have learnt about:

Theorem [Red-Black tree] We can do search, insert, delete, min, max, predecessor, and successor operations in $O(\lg n)$ time.